

Конспект лекции 15: «Матрица». Проблема копирования

Преподаватель: Сергей Шершаков

1. Суть проблемы копирования объектов с ресурсами

Если класс управляет динамической памятью (или другими внешними ресурсами), дефолтное побитовое копирование, которое компилятор генерирует автоматически, приводит к критическим багам.

- **Поверхностное копирование (Shallow Copy):** Поля-указатели копируются «как есть». В результате два независимых объекта начинают указывать на один и тот же адрес динамической памяти в куче.
- **Проблема двойного освобождения (Double Free):** При выходе объектов из области видимости сначала деструктор одного объекта успешно освободит память, а затем деструктор второго попытается удалить тот же адрес памяти, что приводит к аварийному завершению программы (Undefined Behavior / Crash).

2. Конструктор копирования (Copy Constructor)

Чтобы объекты корректно создавали независимые копии данных («глубокое копирование»), необходимо явно реализовать конструктор копирования.

- **Сигнатура:** Он принимает константную ссылку на объект того же типа. Передача по ссылке обязательна, иначе возникнет бесконечная рекурсия при передаче параметров.

```
// В заголовке класса: Matrix(const Matrix& rhv);

Matrix::Matrix(const Matrix& rhv)
    : _rowsNum(rhv._rowsNum)
    , _colsNum(rhv._colsNum)
    , _data(rhv._data ? new double[rhv._rowsNum * rhv._colsNum] : nullptr)
{
    if (_data) {
        for (size_t i = 0; i < _rowsNum * _colsNum; ++i) {
            _data[i] = rhv._data[i]; // Глубокое копирование элементов
        }
    }
}
```

3. Оператор присваивающего копирования (Copy Assignment Operator)

Вызывается при присваивании одного уже существующего объекта другому (например, `m1 = m2;`).

- **Идиома Copy-and-Swap:** Самый безопасный и элегантный способ реализации оператора присваивания. Вместо ручной очистки старой памяти и выделения новой (что небезопасно с точки зрения исключений), мы переиспользуем конструктор копирования и функцию обмена.

```
Matrix& Matrix::operator=(const Matrix& rhv) {  
    if (this != &rhv) { // Проверка на самоприсваивание  
        Matrix temp(rhv); // Переиспользуем конструктор копирования  
        swap(*this, temp); // Обмениваемся ресурсами с временным объектом  
    }  
    return *this; // Возвращаем ссылку на себя для поддержки цепочек  
}
```

4. Функция swap и спецификатор noexcept

Метод обмена должен гарантированно работать без генерации исключений, поэтому его помечают как **noexcept**.

```
void Matrix::swap(Matrix& a, Matrix& b) noexcept {  
    std::swap(a._rowsNum, b._rowsNum);  
    std::swap(a._colsNum, b._colsNum);  
    std::swap(a._data, b._data); // Обмениваемся только указателями  
}
```

5. Правило трёх (Rule of Three) и Правило пяти (Rule of Five)

- **Правило трёх:** Эмпирическое правило C++, гласящее, что если программисту пришлось явно определить хотя бы один из следующих методов:

1. Деструктор
2. Конструктор копирования
3. Оператор присваивающего копирования

то с высокой вероятностью классу **необходимо явно определить все три метода**.

- **Правило пяти:** В современном C++ (начиная с C++11) к этому списку добавляются еще два метода для поддержки семантики перемещения (Move Semantics): *Конструктор перемещения (Move Constructor)* и *Оператор присваивающего перемещения (Move Assignment Operator)*.

6. Запрет копирования

Если логика класса не подразумевает возможность копирования (например, класс управляет уникальным файлом или соединением), возможность копирования можно явно заблокировать компилятору с помощью ключевого слова = *delete*:

```
Matrix(const Matrix& rhv) = delete;  
Matrix& operator=(const Matrix& rhv) = delete;
```